

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence
PRACTICAL NO. 2

Aim:

- Implement the A* Search algorithm for solving a pathfinding problem.
- Implement the Recursive Best-First Search algorithm for the same problem.
- Compare the performance and effectiveness of both algorithms.

CODE:

```
import heapq
import matplotlib.pyplot as plt
import networkx as nx

# Graph Representation
graph = {
    'Powai Lake': {'Andheri': 7, 'Ghatkopar': 6},
    'Andheri': {'Santacruz': 5, 'Bandra': 8},
    'Ghatkopar': {'Kurla': 5},
    'Santacruz': {'Bandra': 4},
    'Kurla': {'Bandra': 7},
    'Bandra': {'Bandstand': 3},
    'Bandstand': {}
}

# Heuristic Values
heuristics = {
    'Powai Lake': 18,
    'Andheri': 11,
    'Ghatkopar': 14,
    'Santacruz': 6,
    'Kurla': 9,
    'Bandra': 3,
    'Bandstand': 0
}

# Coordinates for graph plotting
node_positions = {
    'Powai Lake': (4, 8),
    'Andheri': (2, 6),
    'Ghatkopar': (6, 6),
    'Santacruz': (2, 4),
    'Kurla': (6, 4),
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

```
'Bandra': (4, 2),  
'Bandstand': (4, 0)  
}
```

```
def a_star_search(graph, heuristics, start, goal):
```

```
    # Priority Queue stores:
```

```
    # (f-score, current node, path, cumulative g-score)
```

```
    priority_queue = [(heuristics[start], start, [start], 0)]
```

```
    visited = set()
```

```
    while priority_queue:
```

```
        # Get node with minimum  $f(n) = g(n) + h(n)$ 
```

```
        f_score, current, path, g_score = heapq.heappop(priority_queue)
```

```
        if current in visited:
```

```
            continue
```

```
        visited.add(current)
```

```
        # Goal Test
```

```
        if current == goal:
```

```
            return path, g_score
```

```
        for neighbour, cost in graph[current].items():
```

```
            new_g = g_score + cost
```

```
            new_f = new_g + heuristics[neighbour]
```

```
            heapq.heappush(
```

```
                priority_queue,
```

```
                (new_f, neighbour, path + [neighbour], new_g)
```

```
            )
```

```
    return None, float('inf')
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence
PRACTICAL NO. 2

```
# Start and Goal
start = 'Powai Lake'
goal = 'Bandstand'

optimal_path, total_cost = a_star_search(
    graph,
    heuristics,
    start,
    goal
)

print("Optimal A* Path:")
print(" -> ".join(optimal_path))

print("Total Road Distance:", total_cost, "km")

# Graph Plotting
G = nx.DiGraph()

for node, neighbours in graph.items():
    for neighbour, weight in neighbours.items():
        G.add_edge(node, neighbour, weight=weight)

plt.figure(figsize=(10, 8))

# Highlight optimal path
path_edges = list(zip(optimal_path, optimal_path[1:]))

normal_edges = [
    edge for edge in G.edges()
    if edge not in path_edges
]

# Draw nodes
nx.draw_networkx_nodes(
    G,
    node_positions,
    node_color='skyblue',
    node_size=1000
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

)

Draw normal edges

```
nx.draw_networkx_edges(  
    G,  
    node_positions,  
    edgelist=normal_edges,  
    width=1.5,  
    edge_color='gray',  
    arrows=True  
)
```

Draw optimal path

```
nx.draw_networkx_edges(  
    G,  
    node_positions,  
    edgelist=path_edges,  
    width=3.5,  
    edge_color='darkorange',  
    arrows=True  
)
```

Node labels with heuristic

```
node_labels = {  
    node: f"{node}\nh(n)={heuristics[node]}"  
    for node in G.nodes()  
}  
}
```

nx.draw_networkx_labels(
 G,
 node_positions,
 labels=node_labels,
 font_size=9,
 font_weight='bold'
)

Edge labels

```
edge_labels = nx.get_edge_attributes(G, 'weight')
```

```
edge_labels = {
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

```
k: f"{v} km"
for k, v in edge_labels.items()
}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
    font_color='red'
)

plt.title(
    "A* Routing Map: Powai Lake to Bandstand\n"
    "(Highlighted Path = Optimal Route)",
    fontsize=13
)

plt.axis("off")
plt.tight_layout()
plt.show()
```

CODE:

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

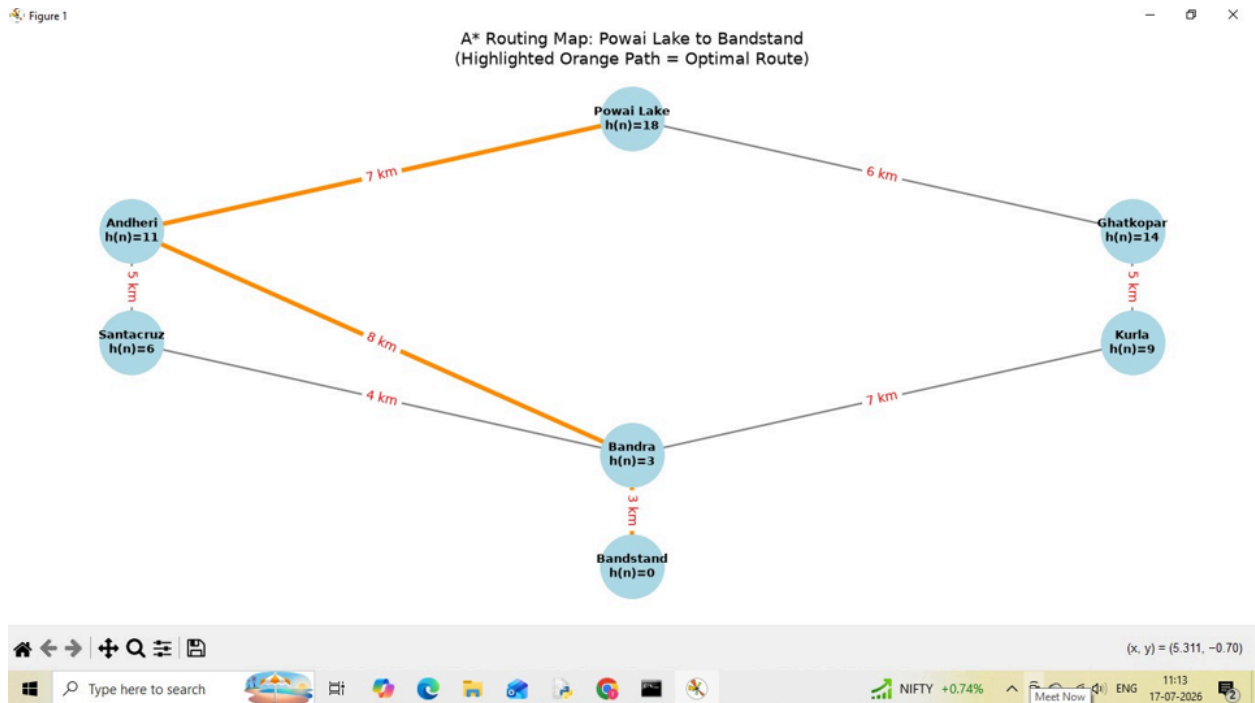
PRACTICAL NO. 2

OUTPUT:

```
T071_AI_Pracno_2.py - C:/...
File Edit Format Run
# Node Labels wi
node_labels = {
    node: f"(nod
for node in
}
nx.draw_networkx
G,
node_positic
labels=node_
font_size=9,
font_weight=
)
# Edge Labels
edge_labels = nx
edge_labels = {k
nx.draw_networkx
G,
node_positic
edge_labels=
font_color="
)
plt.title(
"A* Routing
"(Highlighte
fontsize=13
)
plt.axis("off")
plt.tight_layout
plt.show()
```

```
IDLE Shell 3.14.6
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py =====
Optimal Path Discovered:
Powai Lake -> Andheri -> Bandra -> Bandstand

Total Road Distance: 18 km
```



SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

CODE:

```
import matplotlib.pyplot as plt
```

```
# Graph Representation
```

```
graph = {  
    'Powai Lake': {'Andheri': 7, 'Ghatkopar': 6},  
    'Andheri': {'Santacruz': 5, 'Bandra': 8},  
    'Ghatkopar': {'Kurla': 5},  
    'Santacruz': {'Bandra': 4},  
    'Kurla': {'Bandra': 7},  
    'Bandra': {'Bandstand': 3},  
    'Bandstand': {}  
}
```

```
# Heuristic Values
```

```
heuristics = {  
    'Powai Lake': 18,  
    'Andheri': 11,  
    'Ghatkopar': 14,  
    'Santacruz': 6,  
    'Kurla': 9,  
    'Bandra': 3,  
    'Bandstand': 0  
}
```

```
# Coordinates for graph plotting
```

```
coords = {  
    'Powai Lake': (4, 8),  
    'Andheri': (2, 6),  
    'Ghatkopar': (6, 6),  
    'Santacruz': (2, 4),  
    'Kurla': (6, 4),  
    'Bandra': (4, 2),  
    'Bandstand': (4, 0)  
}
```

```
def rbfs_search(start, goal):
```

```
    success, path, cost, _ = rbfs(
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence
PRACTICAL NO. 2

```
start,  
goal,  
0,  
float("inf"),  
[start]  
)
```

```
return path, cost
```

```
def rbfs(node, goal, g, f_limit, path):
```

```
# Goal Test  
if node == goal:  
    return True, path, g, g
```

```
successors = []
```

```
# Generate successors  
for neighbour, distance in graph[node].items():
```

```
    if neighbour not in path:
```

```
        next_g = g + distance
```

```
        next_f = max(  
            next_g + heuristics[neighbour],  
            g + heuristics[node]  
        )
```

```
        successors.append(  
            [next_f, neighbour, next_g]  
        )
```

```
# No successors  
if not successors:  
    return False, [], 0, float("inf")
```

```
while True:
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

```
# Sort according to f-value
```

```
successors.sort(key=lambda x: x[0])
```

```
best = successors[0]
```

```
# Best option exceeds current f-limit
```

```
if best[0] > f_limit:
```

```
    return False, [], 0, best[0]
```

```
# Find alternative path
```

```
alternative_f = (
```

```
    successors[1][0]
```

```
    if len(successors) > 1
```

```
    else float("inf")
```

```
)
```

```
success, result_path, total_g, returned_f = rbfs(
```

```
    best[1],
```

```
    goal,
```

```
    best[2],
```

```
    min(f_limit, alternative_f),
```

```
    path + [best[1]]
```

```
)
```

```
best[0] = returned_f
```

```
if success:
```

```
    return True, result_path, total_g, returned_f
```

```
# Start and Goal
```

```
path, total_dist = rbfs_search(
```

```
    'Powai Lake',
```

```
    'Bandstand'
```

```
)
```

```
print(
```

```
    "Optimal RBFS Path:",
```

```
    " -> ".join(path)
```

```
)
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence
PRACTICAL NO. 2

```
print(
    "Total Distance:",
    total_dist,
    "km"
)

# Graph Plotting
plt.figure(figsize=(8, 7))

for node, neighbours in graph.items():

    x1, y1 = coords[node]

    for neighbour, dist in neighbours.items():

        x2, y2 = coords[neighbour]

        is_path = (
            node in path and
            neighbour in path and
            path.index(neighbour) == path.index(node) + 1
        )

        if is_path:
            line_color = 'darkorange'
            line_width = 3
        else:
            line_color = 'lightblue'
            line_width = 1.5

        plt.plot(
            [x1, x2],
            [y1, y2],
            color=line_color,
            linewidth=line_width
        )

    plt.text(
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

```
(x1 + x2) / 2,  
(y1 + y2) / 2,  
f"{dist} km",  
color='red',  
fontsize=9,  
ha='center'  
)
```

Draw nodes

for node, (x, y) in coords.items():

if node in path:

node_color = '#f1c40f'

else:

node_color = '#3498db'

plt.scatter(
x,
y,
color=node_color,
s=700
)

plt.text(
x,
y,
f"{node}\nh={heuristics[node]}",
ha='center',
va='center',
color='white',
fontsize=9,
fontweight='bold'
)

plt.title(
f"RBFS Shortest Route Map\n"
f"Powai Lake to Bandstand (Total: {total_dist} km)",
fontsize=12,

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

fontweight='bold'

)

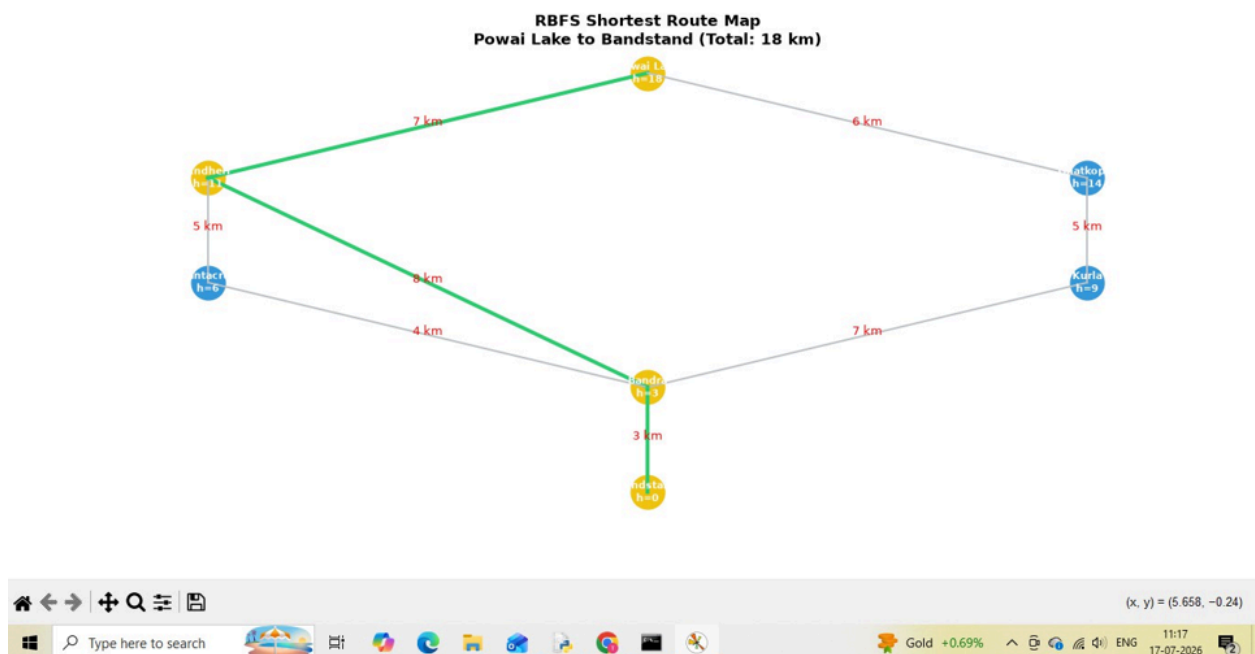
plt.axis("off")

plt.show()

OUTPUT:

```
T071_AI_Pracno_2.py  IDLE Shell 3.14.6
File Edit Format Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py =====
Optimal RBFS Path: Powai Lake -> Andheri -> Bandra -> Bandstand (18 km)
```

Figure 1



SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

##
dict_hn = {
    'Powai Lake': 18,
    'Andheri': 11,
    'Ghatkopar': 14,
    'Santacruz': 6,
    'Kurla': 9,
    'Bandra': 3,
    'Bandstand': 0
}

dict_gn = dict(
    **{
        'Powai Lake': dict(Andheri=7, Ghatkopar=6),
        'Andheri': dict(Powai_Lake=7, Santacruz=5, Bandra=8),
        'Ghatkopar': dict(Powai_Lake=6, Kurla=5),
        'Santacruz': dict(Andheri=5, Bandra=4),
        'Kurla': dict(Ghatkopar=5, Bandra=7),
        'Bandra': dict(Andheri=8, Santacruz=4, Kurla=7, Bandstand=3),
        'Bandstand': dict()
    }
)

# Rename key to match dictionary
dict_gn['Andheri']['Powai Lake'] = dict_gn['Andheri'].pop('Powai_Lake')
dict_gn['Ghatkopar']['Powai Lake'] = dict_gn['Ghatkopar'].pop('Powai_Lake')

import queue as Q

start = 'Powai Lake'
goal = 'Bandstand'
result = ''

def get_fn(citystr):
    cities = citystr.split(" , ")
    hn = gn = 0

Ln: 602 Col: 6
```

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

##
dict_hn = {
    'Powai Lake': 18,
    'Andheri': 11,
    'Ghatkopar': 14,
    'Santacruz': 6,
    'Kurla': 9,
    'Bandra': 3,
    'Bandstand': 0
}

dict_gn = dict(
    **{
        'Powai Lake': dict(Andheri=7, Ghatkopar=6),
        'Andheri': dict(Powai_Lake=7, Santacruz=5, Bandra=8),
        'Ghatkopar': dict(Powai_Lake=6, Kurla=5),
        'Santacruz': dict(Andheri=5, Bandra=4),
        'Kurla': dict(Ghatkopar=5, Bandra=7),
        'Bandra': dict(Andheri=8, Santacruz=4, Kurla=7, Bandstand=3),
        'Bandstand': dict()
    }
)

# Rename key to match dictionary
dict_gn['Andheri']['Powai Lake'] = dict_gn['Andheri'].pop('Powai_Lake')
dict_gn['Ghatkopar']['Powai Lake'] = dict_gn['Ghatkopar'].pop('Powai_Lake')

import queue as Q

start = 'Powai Lake'
goal = 'Bandstand'
result = ''

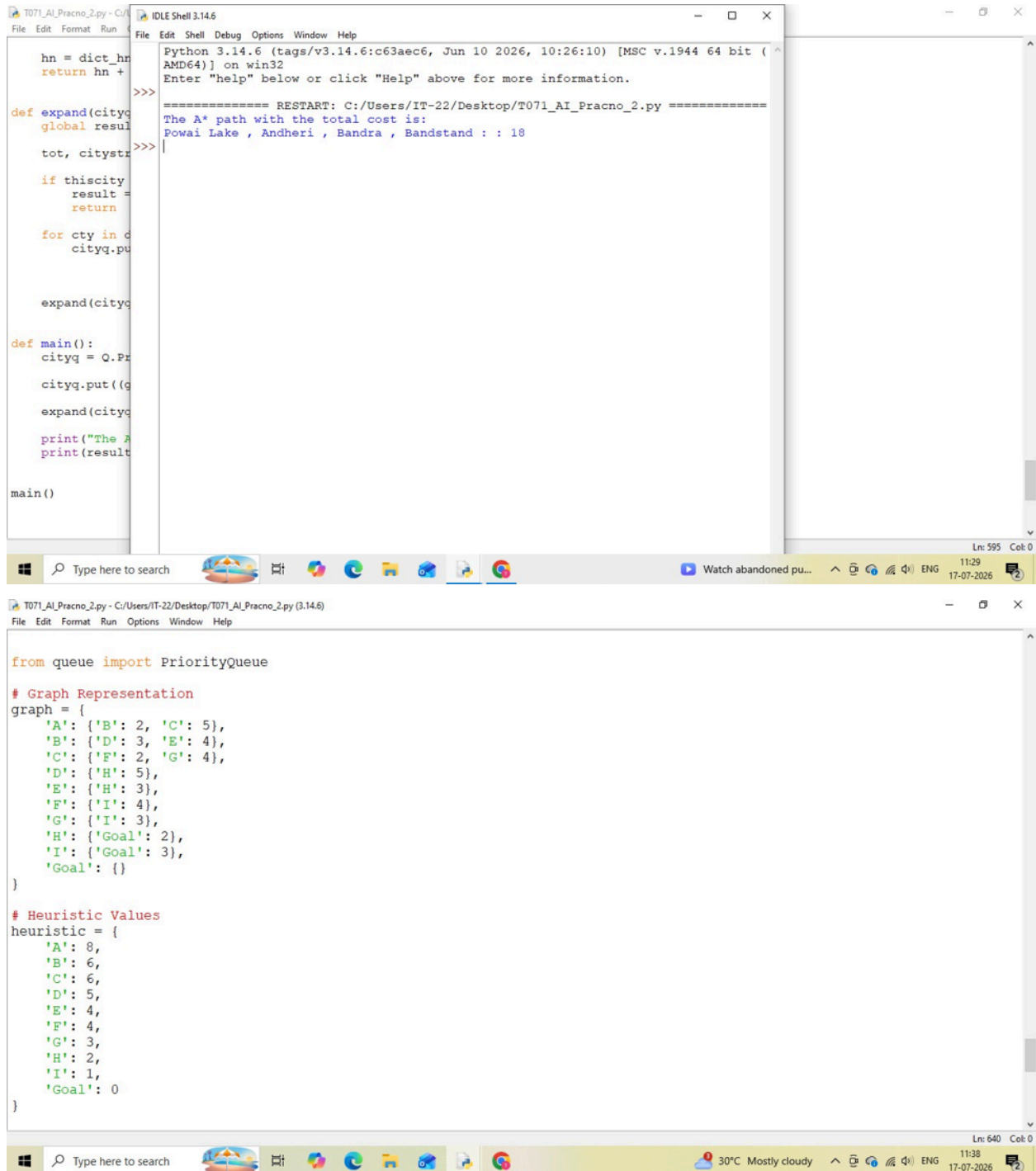
def get_fn(citystr):
    cities = citystr.split(" , ")
    hn = gn = 0

Ln: 602 Col: 6
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2



The image shows two screenshots of a Python IDE (IDLE 3.14.6) running a program. The top screenshot shows the initial execution of a script named 'T071_AI_Pracno_2.py'. The code defines a function 'expand' and a 'main' function. The output in the shell window shows a restart message and the result: 'The A* path with the total cost is: Powai Lake , Andheri , Bandra , Bandstand : : 18'. The bottom screenshot shows the same IDE with a different script, 'T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)'. This script defines a graph and heuristic values for a pathfinding algorithm. The graph is represented as a dictionary where keys are nodes and values are dictionaries of neighbors and their costs. The heuristic values are also represented as a dictionary.

```
from queue import PriorityQueue

# Graph Representation
graph = {
    'A': {'B': 2, 'C': 5},
    'B': {'D': 3, 'E': 4},
    'C': {'F': 2, 'G': 4},
    'D': {'H': 5},
    'E': {'H': 3},
    'F': {'I': 4},
    'G': {'I': 3},
    'H': {'Goal': 2},
    'I': {'Goal': 3},
    'Goal': {}
}

# Heuristic Values
heuristic = {
    'A': 8,
    'B': 6,
    'C': 6,
    'D': 5,
    'E': 4,
    'F': 4,
    'G': 3,
    'H': 2,
    'I': 1,
    'Goal': 0
}
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

start = 'A'
goal = 'Goal'
# Priority Queue stores (f, g, path)
pq = PriorityQueue()
pq.put((heuristic[start], 0, [start]))

visited = set()

while not pq.empty():
    f, g, path = pq.get()
    node = path[-1]

    if node == goal:
        print("\nBest Path :", " -> ".join(path))
        print("Total Cost =", g)
        break

    if node in visited:
        continue

    visited.add(node)

    for neighbour, cost in graph[node].items():
        new_g = g + cost
        new_f = new_g + heuristic[neighbour]

        print(
Ln: 640 Col: 0
30°C Mostly cloudy 11:38 17-07-2026
```

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

    if node == goal:
        print("\nBest Path :", " -> ".join(path))
        print("Total Cost =", g)
        break

    if node in visited:
        continue

    visited.add(node)

    for neighbour, cost in graph[node].items():
        new_g = g + cost
        new_f = new_g + heuristic[neighbour]

        print(
            path + [neighbour],
            "g =", new_g,
            "h =", heuristic[neighbour],
            "f =", new_f
        )

        pq.put((new_f, new_g, path + [neighbour]))
Ln: 640 Col: 0
30°C Mostly cloudy 11:38 17-07-2026
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help
)
# Node Labels with Heuristics
node_labels = {
    node: f"{node}\nh(n)={heuristics[node]}"
    for node in G.nodes()
}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels=node_labels,
    font_size=9,
    font_weight="bold"
)

# Edge Labels
edge_labels = nx.get_edge_attributes(G, "weight")
edge_labels = {k: f"{v} km" for k, v in edge_labels.items()}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
    font_color="red"
)

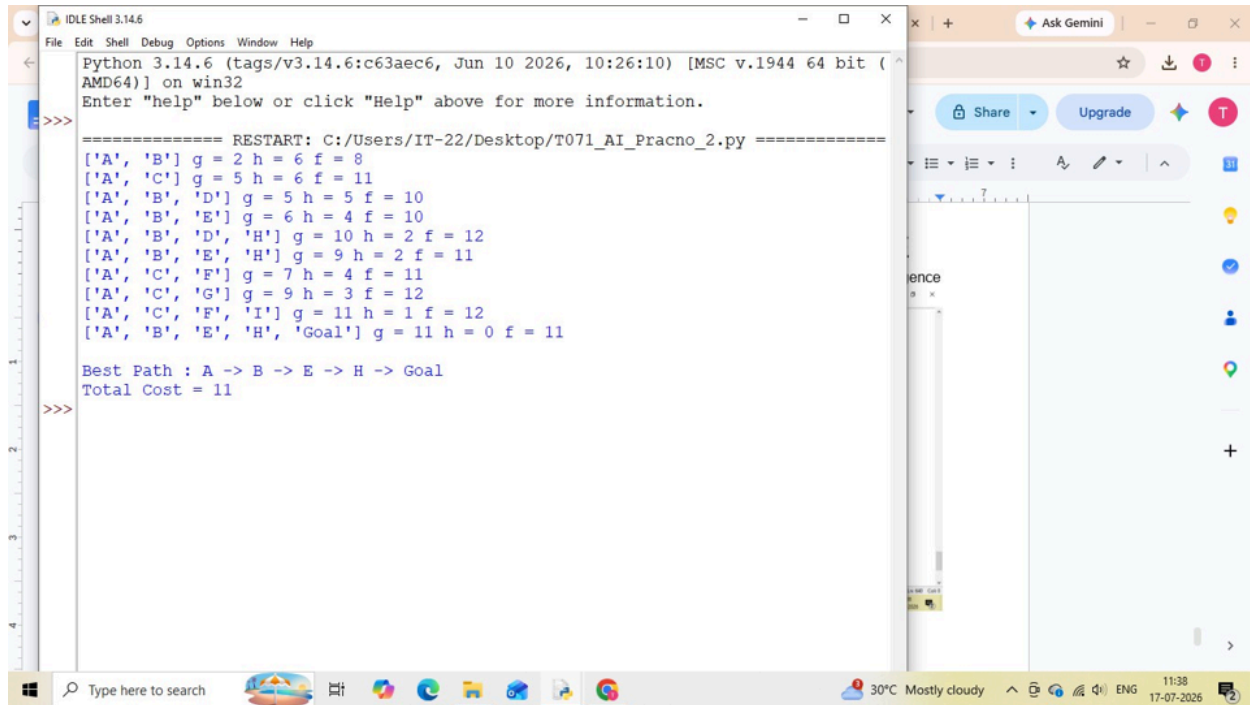
plt.title(
    "A* Routing Map: Powai Lake to Bandstand\n"
    "(Highlighted Orange Path = Optimal Route)",
    fontsize=13
)

plt.axis("off")
plt.tight_layout()
plt.show()
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT: Artificial intelligence

PRACTICAL NO. 2



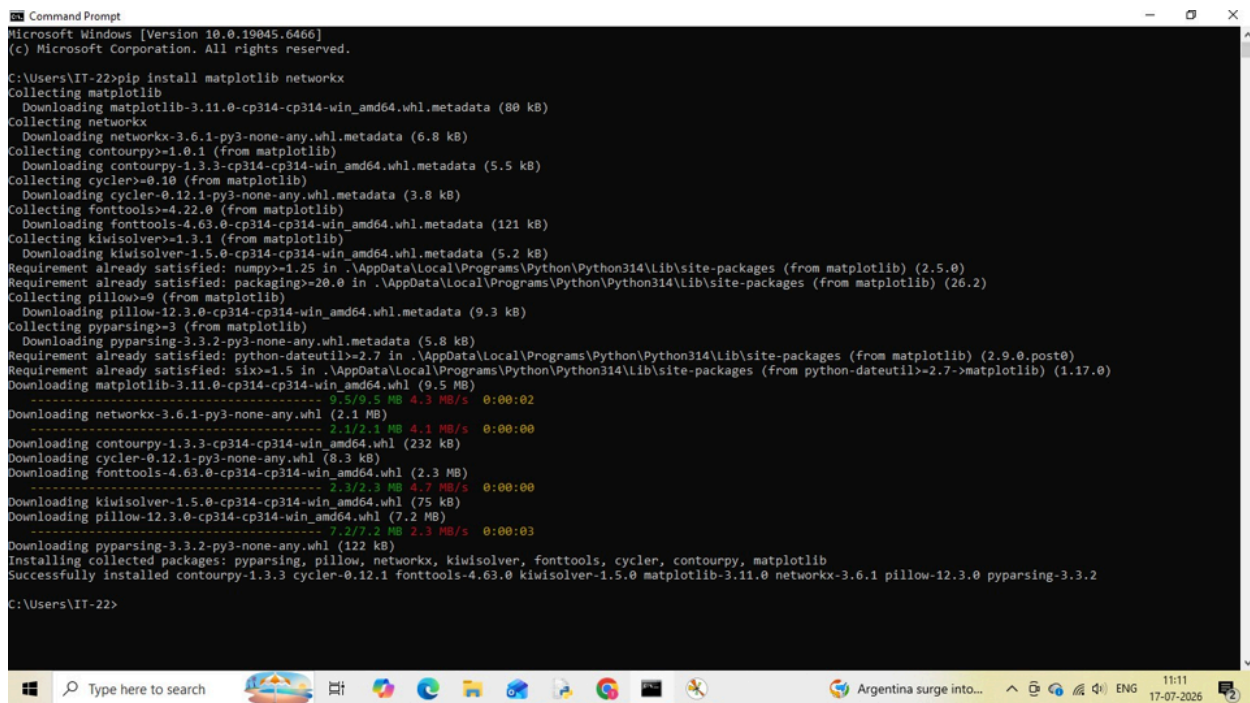
```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>

===== RESTART: C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py =====
['A', 'B'] g = 2 h = 6 f = 8
['A', 'C'] g = 5 h = 6 f = 11
['A', 'B', 'D'] g = 5 h = 5 f = 10
['A', 'B', 'E'] g = 6 h = 4 f = 10
['A', 'B', 'D', 'H'] g = 10 h = 2 f = 12
['A', 'B', 'E', 'H'] g = 9 h = 2 f = 11
['A', 'C', 'F'] g = 7 h = 4 f = 11
['A', 'C', 'G'] g = 9 h = 3 f = 12
['A', 'C', 'F', 'I'] g = 11 h = 1 f = 12
['A', 'B', 'E', 'H', 'Goal'] g = 11 h = 0 f = 11

Best Path : A -> B -> E -> H -> Goal
Total Cost = 11

>>>
```



```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\IT-22>pip install matplotlib networkx
Collecting matplotlib
  Downloading matplotlib-3.11.0-cp314-cp314-win_amd64.whl.metadata (80 kB)
Collecting networkx
  Downloading networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp314-cp314-win_amd64.whl.metadata (5.5 kB)
Collecting cyclor>=0.10 (from matplotlib)
  Downloading cyclor-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.63.0-cp314-cp314-win_amd64.whl.metadata (121 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.0-cp314-cp314-win_amd64.whl.metadata (5.2 kB)
Requirement already satisfied: numpy>=1.25 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.5.0)
Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (26.2)
Collecting pillow>=9 (from matplotlib)
  Downloading pillow-12.3.0-cp314-cp314-win_amd64.whl.metadata (9.3 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Download matplotlib-3.11.0-cp314-cp314-win_amd64.whl (9.5 MB)
----- 9.5/9.5 MB 4.3 MB/s 0:00:02
Download networkx-3.6.1-py3-none-any.whl (2.1 MB)
----- 2.1/2.1 MB 4.1 MB/s 0:00:00
Download contourpy-1.3.3-cp314-cp314-win_amd64.whl (232 kB)
Download cyclor-0.12.1-py3-none-any.whl (8.3 kB)
Download fonttools-4.63.0-cp314-cp314-win_amd64.whl (2.3 MB)
----- 2.3/2.3 MB 4.7 MB/s 0:00:00
Download kiwisolver-1.5.0-cp314-cp314-win_amd64.whl (75 kB)
Download pillow-12.3.0-cp314-cp314-win_amd64.whl (7.2 MB)
----- 7.2/7.2 MB 2.3 MB/s 0:00:03
Download pyparsing-3.3.2-py3-none-any.whl (122 kB)
Installing collected packages: pyparsing, pillow, networkx, kiwisolver, fonttools, cyclor, contourpy, matplotlib
Successfully installed contourpy-1.3.3 cyclor-0.12.1 fonttools-4.63.0 kiwisolver-1.5.0 matplotlib-3.11.0 networkx-3.6.1 pillow-12.3.0 pyparsing-3.3.2

C:\Users\IT-22>
```